

Unit-2: Generative Models for Text

Course Outcomes & Syllabus

CO2: Differentiate generative and discriminative models using probabilistic and architectural perspectives.

Syllabus:-Generative Models for Text: Language Models Basics, building blocks of Language models, Transformer Architecture, Encoder and Decoder, Attention mechanisms, Generation of Text, Models like BERT and GPT models, Generation of Text, Auto encoding, Regression Models, Exploring Chat GPT, Prompt Engineering: Designing Prompts, Revising Prompts using Reinforcement Learning from Human Feedback (RLHF), Retrieval Augmented Generation, Multimodal LLM, Issues of LLM like hallucination.

T1: Denis Rothman, “Transformers for Natural Language Processing and Computer Vision”, Third Edition, Packt Books, 2024

R1: David Foster,” Generative Deep Learning”, O’Reily Books, 2024.

R2: Altaf Rehmani, “Generative AI for Everyone”, BlueRose One, 2024

Learning Outcomes

At the end of this lecture, Student will be able to:

LO1: Explain the architecture and functioning of language models, Transformers, Attention Mechanisms, BERT and GPT for text generation.

LO2: Apply prompt engineering, RLHF, and Retrieval-Augmented Generation techniques to enhance large language model performance.

LO3: Analyze multimodal LLMs, generative modeling approaches, and challenges such as hallucination in AI-generated content..

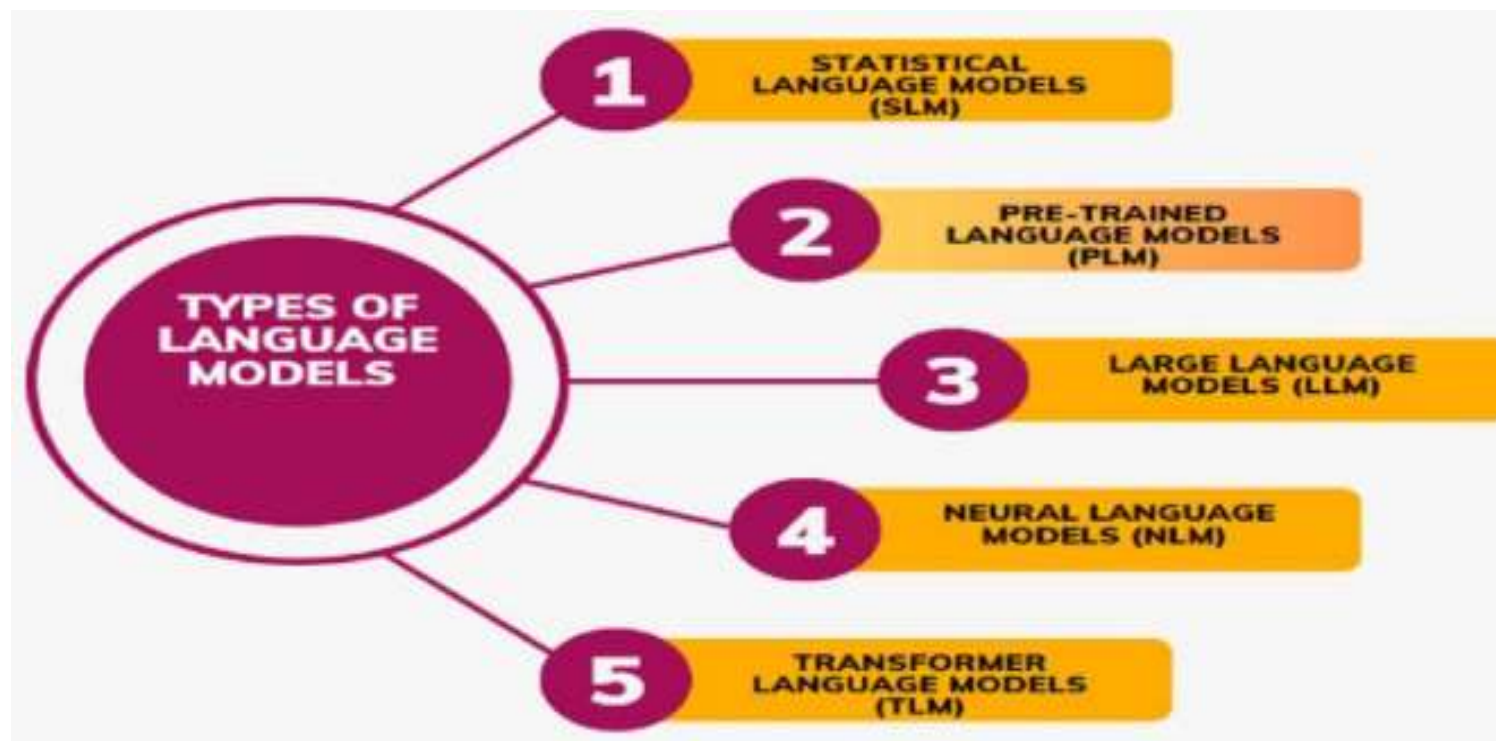
Language Models Basics

- ❖ Generative Models for Text are a class of models in AI/ML which learn the probability distribution of language and can generate new text that resembles human writing.
- ❖ Generative models for text are AI systems, primarily based on Transformer architectures (like GPT, BERT, T5), that learn patterns from vast datasets to create new, coherent human-like text.
- ❖ These models, often using decoder-only structures for autoregressive, next-token prediction, enable applications such as automated content creation, translation, summarization, and interactive chatbots.
- ❖ Generative models for text, primarily Large Language Models (LLMs), are a subset of artificial intelligence designed to create new, human-like text by learning patterns from massive datasets.

Language Models Basics

- ❖ A language model (LM) is a fundamental concept in Natural Language Processing (NLP) that learns how words are arranged in a language and predicts the likelihood of word sequences.
- ❖ A language model (LM) is a fundamental concept in Natural Language Processing (NLP) that learns how words are arranged in a language and predicts the likelihood of word sequences.

Types of Language Models



Language Models Basics

1. Statistical Language Models[SLM] (n-grams): These calculate probabilities based on a limited number of previous **words** (e.g., predicting the next word based on only the last two). There are different types of n-gram models such as,

- Unigrams that evaluate each word independently;
- Bigrams that consider the probability of a word given the previous word;
- Trigrams that consider the probability of a word given the two previous words; and soon

2. Pre-Trained Language Models[PLM]-Pre-trained language models (PLMs) are dominating in the NLP field nowadays.

- With the development of deep learning, the training and usage of PLMs have changed a lot as compared with conventional statistical LMs.

3. Large Language Models[LLM]-Large language models are typically trained on massive amounts of text data, such as the entire internet or large collections of books, articles, and other written material.

Language Models Basics

4. Neural Language Models [NLM]: These are used in deep learning (e.g., Transformers) to capture long-range dependencies and complex linguistic structures.

- Neural language models, as the name suggests, use neural networks to predict the likelihood of a sequence of words.

5. Transformer Language Models [TLM] - Transformer language models revolutionized language modeling by introducing a novel self attention mechanism that effectively captures long-range inter dependence in texts.

Applications of Language Models:

Text Generation

Chat bots & Virtual Assistants

Machine Translation

Text Summarization

Speech Recognition

Auto complete & Next Word Prediction

Social Media Monitoring

Healthcare & Legal Applications

Building blocks of Language models:

- Language models are built from a set of core components that work together to understand and generate text.
- Language models are built on tokens, embeddings, and vectors, which process text through neural network layers to understand and generate language.
- These models, typically using transformer architecture, function through input (tokens), processing (attention mechanisms), and output (probabilistic next-token generation) stages.

Building blocks of language model include:

Tokens: The fundamental units of data (words, sub words, or characters) that the model processes. Text is broken down into these small pieces, which are converted into unique IDs.

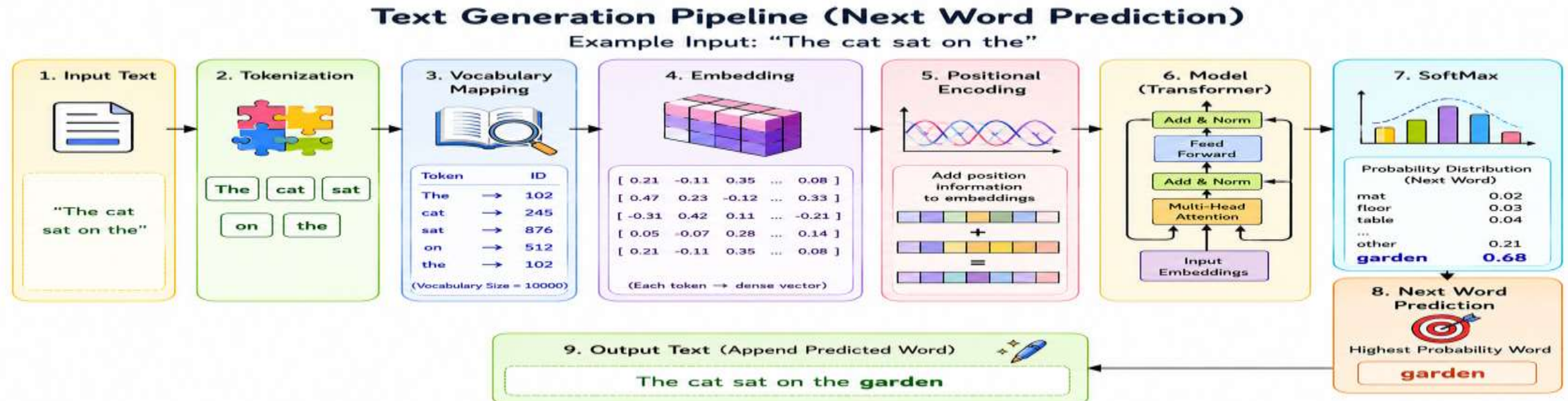
Embeddings: A numerical representation that maps tokens into a continuous, high-dimensional vector space, capturing semantic meaning.

Building blocks of Language models:

Vectors: Numerical forms (arrays of numbers) representing tokens or text, which enable the model to perform mathematical operations to understand context.

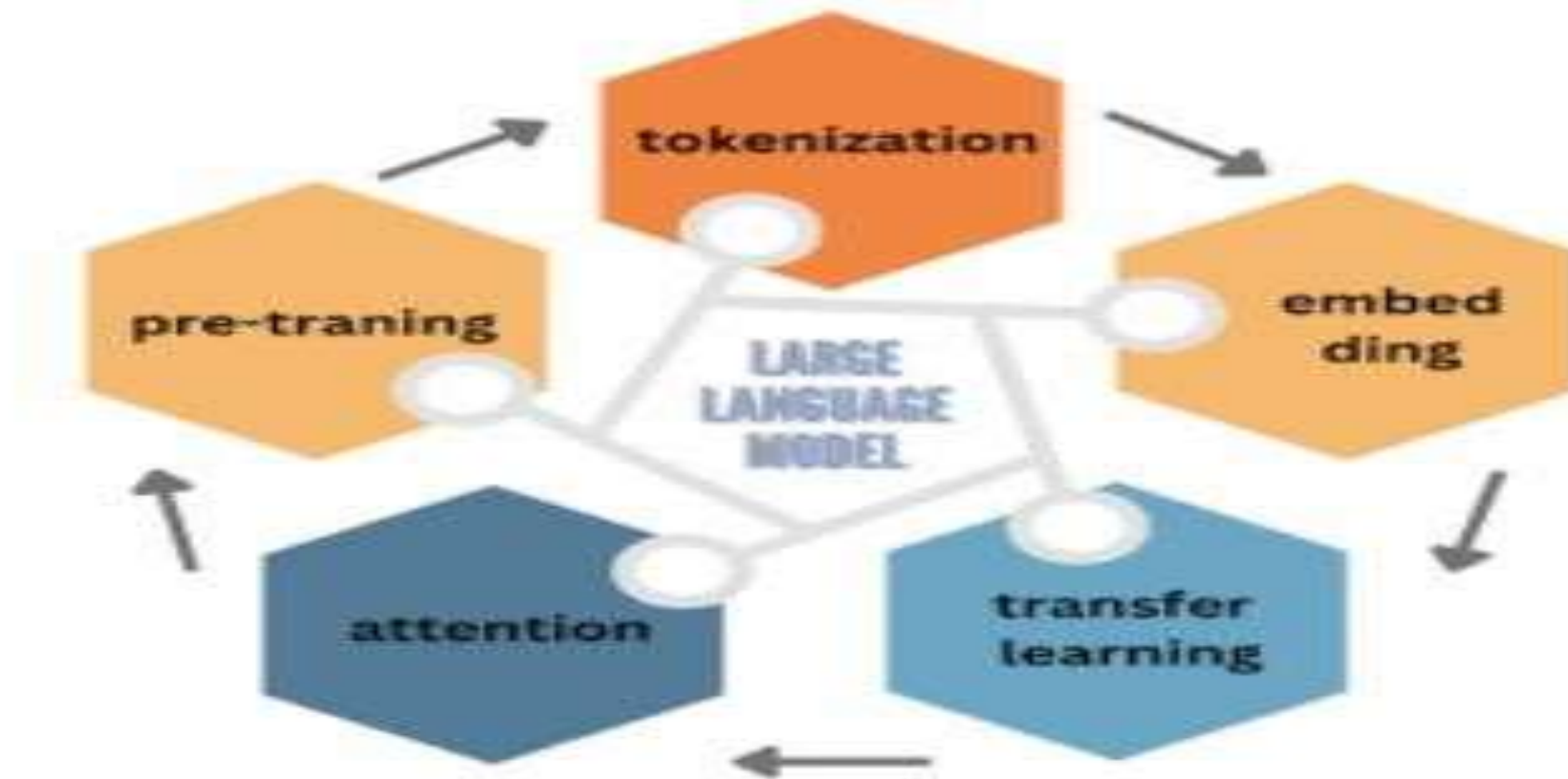
Transformer Architecture: The underlying neural network structure, particularly focusing on self-attention mechanisms that determine how words in a sentence relate to each other.

Parameters: Weights and biases within the neural network that are updated during training to improve accuracy.



Building blocks of Language models:

BASIC BUILDING BLOCKS OF LLM



Building blocks of Language models:

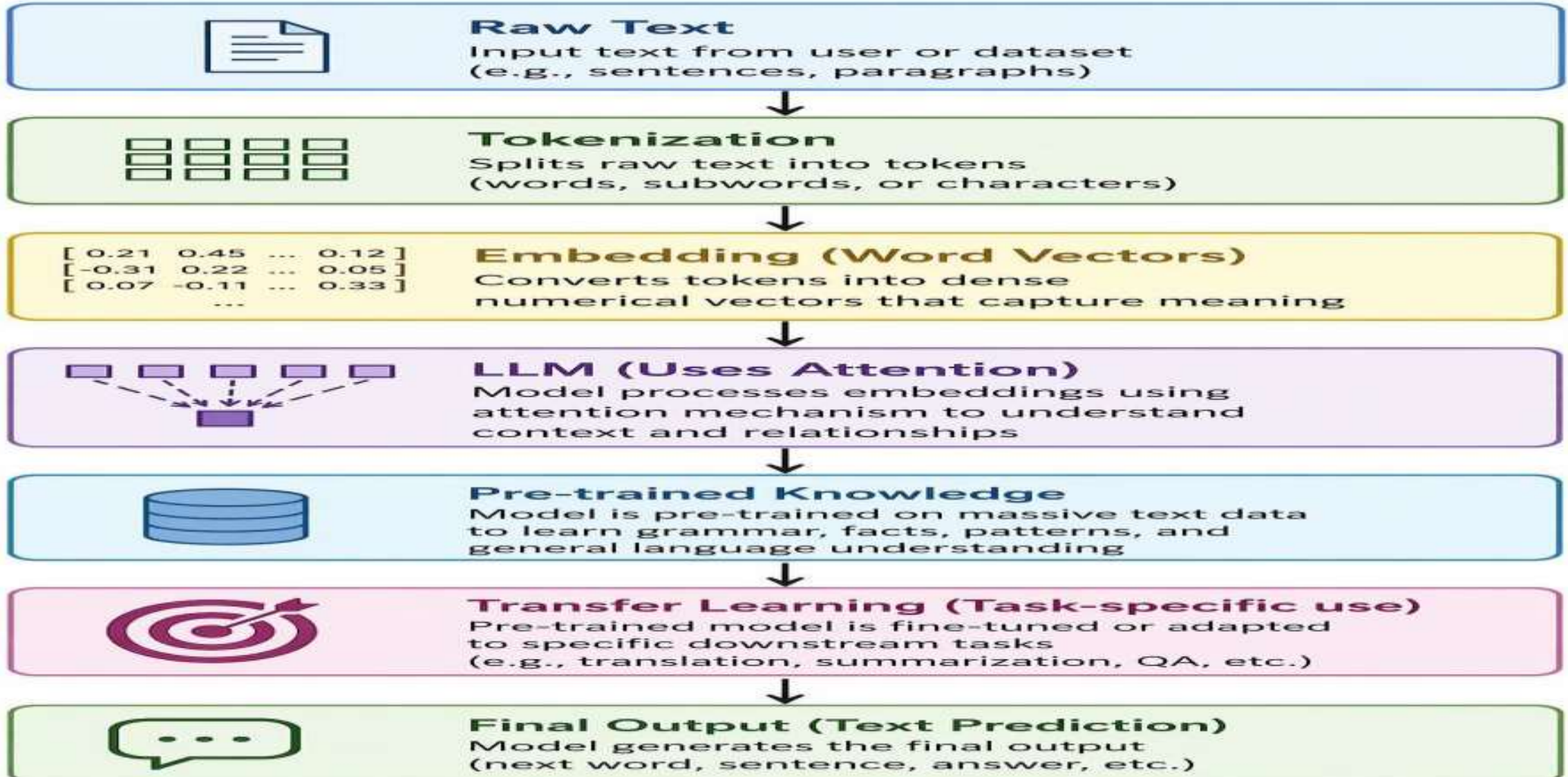
From the above diagram at the center is the Large Language Model(LLM), and around it are the key components that make it work.

- **Tokenization**-Converts input text into tokens(small units),Tokens can be words, sub words, or characters Example: “AI is powerful” → ["AI", "is", "powerful"].
- **Embedding**-Converts tokens into numerical vectors. These vectors capture meaning and relationships. Example: “king” and “queen” → similar vectors.
- **Attention**-Helps the model focus on important words in a sentence. Assigns different importance (weights) to words
Example:“The cat sat on the mat”→connects cat with sat.

Building blocks of Language models:

- **Transfer Learning**-Model is pre-trained on large data and then adapted. Reused for multiple tasks. Example: Same model used for translation, Chatbot, summarization.
- **Pre-training**-Initial training on massive text data sets. Learns grammar, facts and language patterns. Example: Training on books, websites, articles.
- The overall process—from tokenization and embedding to attention-based processing, pre-training and transfer learning—ensures that the model captures both the structure and meaning of text.
- The following diagram illustrates the step-by-step flow of how a language model processes text and generates output.

Building blocks of Language models:

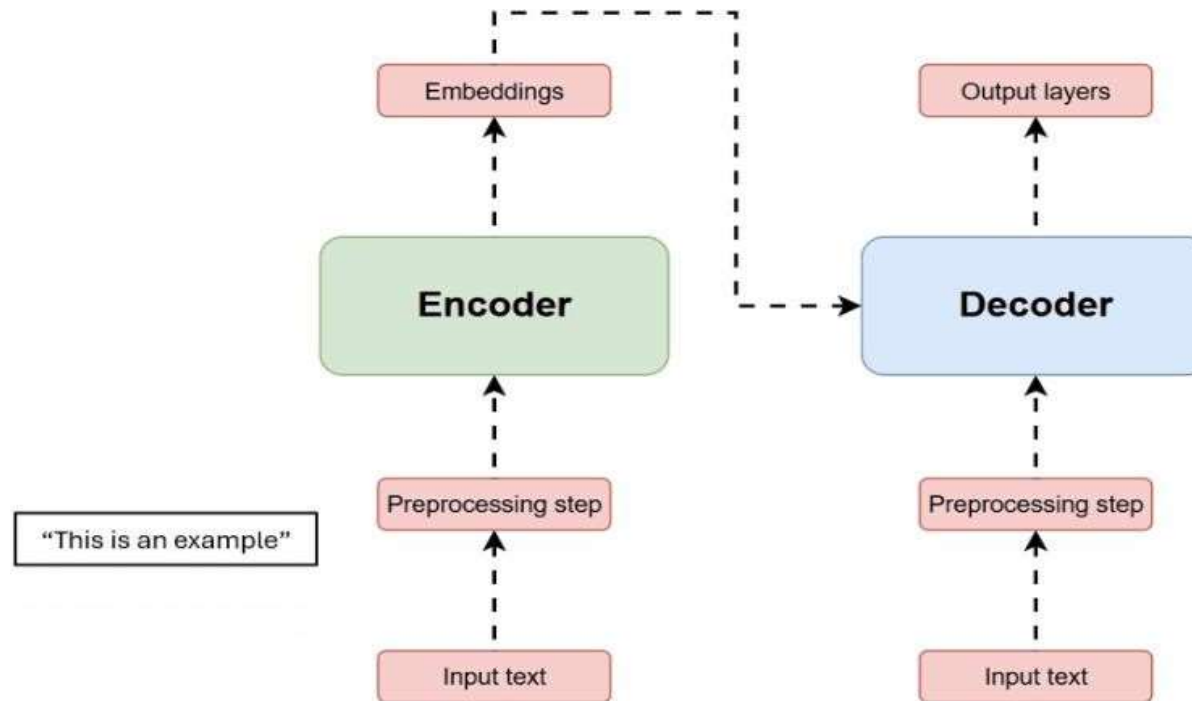


Transformer Architecture

- The Transformer architecture is a powerful deep learning model used for processing sequential data, especially in Natural Language Processing (NLP).
- The Transformer **architecture** is the back bone of modern **Generative AI**, especially for text generation. Models like GPT are built on this architecture to generate human-like text efficiently.
- Encoders and decoders are fundamental neural network components in generative AI that compress input data into a meaningful latent representation (encoding) and subsequently reconstruct or generate new, similar data from that representation (decoding).
- Encoders capture core features and patterns, while decoders interpret the same features to generate text, images, or audio.

Transformer Architecture

- The **Transformer architecture** is a deep learning model used in **Natural Language Processing(NLP)** for understanding and generating text.
- It consists of two main components : **Encoder** and **Decoder** and is widely used in modern models like GPT.
- The below diagram shows how input text is processed through two main components: **Encoder** and **Decoder**, which are the core parts of the Transformer model.



Encoder and Decoder

- **Encoder:** Maps input data(e.g., text, images) into a compressed, high-dimensional representation known as the latent space or vector representation.
- It focuses on capturing the underlying structure and context, often utilizing self-attention layers to identify relationships between data components.
- **Decoder:** Takes the encoded representation from the encoder and reconstructs the data, or generates new content.
- In transformer models, it often includes an additional encoder-decoder attention layer to focus on specific parts of the input, enabling high-quality, relevant outputs.
- The Transformer architecture efficiently processes input text using encoder-decoder structure and attention mechanisms to generate accurate and context-aware outputs.
- The process begins with **input text**, which is preprocessed and converted into tokens.

Encoder and Decoder

- These tokens are then transformed in to **embeddings(numerical vectors)**.
- The **encoder** processes these embeddings using a **self-attention mechanism** and feed-forward layers to capture the relationships between words and generate a contextual representation.
- **Applications:**
 - Machine translation
 - Chat bots
 - Text summarization
 - Question answering
 - Code generation

Attention mechanisms:

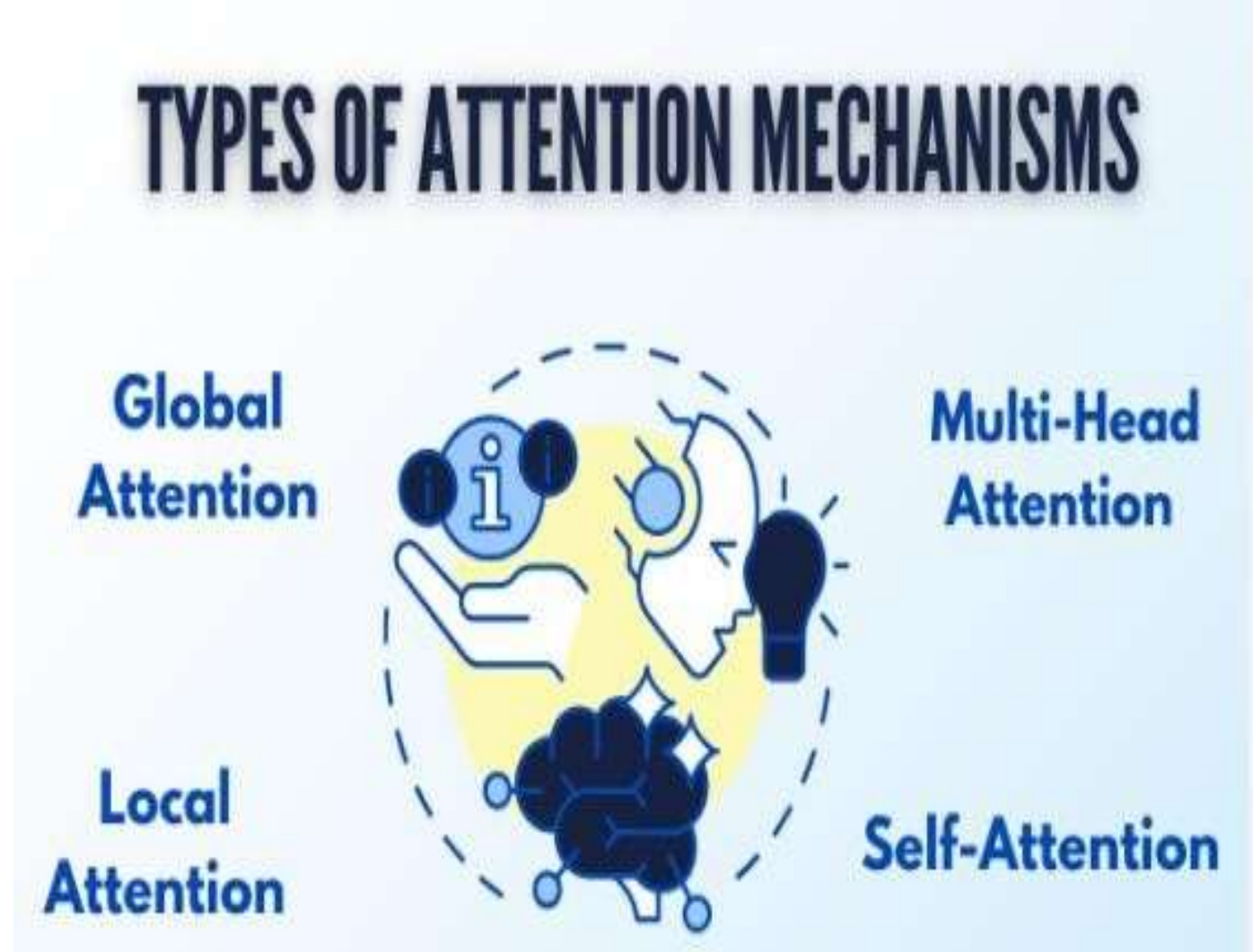
- Attention mechanisms are a key concept in modern NLP that allow models to focus on the most relevant parts of a sentence while processing or generating text.
- “Attention mechanism is a technique that allows a model to focus on important parts of the input sequence by assigning different weights to different words.”
- The **attention mechanism** allows a model to focus on the most relevant words in a sentence while processing text.
- The attention mechanism is a core AI technique allowing models to selectively focus on specific parts of input data, enabling understanding of context and long-range dependencies in generative AI.
- It calculates weights for input elements based on relevance to a query, prioritizing crucial data, particularly crucial for Transformer-based architectures.
- An attention mechanism is a [machine learning](#) technique that directs [deep learning](#) models to prioritize (or *attend to*) the most relevant parts of input data

Attention mechanisms:

- Instead of treating all words equally, attention assigns **different importance(weights)** to each word based on context.
- For Example, let us Observe the Sentence: “The dog chased the ball because it was fast.” here the Attention helps to identify whether “it” refers to dog or ball.
- Attention mechanism is Needed to Handles long sentences, captures relationships between distant words, improves context understanding and Overcomes limitations of RNNs.
- Types of Attention mechanism- [Self-Attention](#) (internal relationships), [Multi-Head Attention](#) (parallel focus), [Soft/Hard Attention](#) (weighting methods), and [Global/Local Attention](#) (context scope).
- Attention mechanisms in deep learning, particularly within transformers, allow models to selectively focus on relevant input parts, improving performance in tasks like NLP and computer vision.

Attention mechanisms Applications & Types

- Applications:
- Machine translation
- Text summarization
- Chat bots
- Question answering
- Sentiment analysis
- Speech recognition Image captioning
- Text generation



Generation of Text

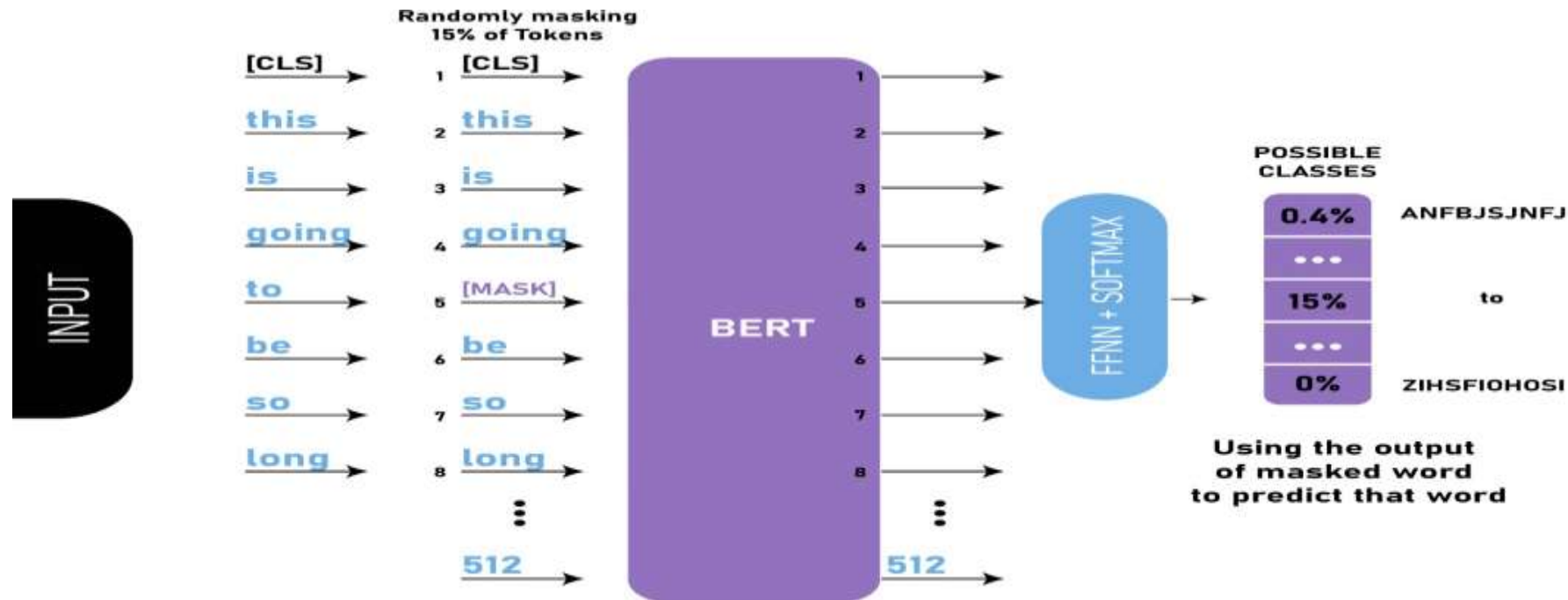
- Text generation is the process by which a model automatically produces meaningful and human-like text based on a given input (prompt).
- Text generation is the process of producing human-like text using AI models such as GPT based on the Transformer architecture.
- A language model generates text by predicting the **nextword(token)** based on previous words.
- For example, let us take **input Text** “Artificial intelligence is "then the predicting the **next Output Text**” transforming the world”.
- The Applications of Text generation are **Content writing, Chat bots, Story generation, Code-generation and Email drafting.**
- **BERT and GPT** are powerful Transformer-based models widely used in **Natural Language Processing(NLP)**. They both rely on attention mechanisms but serve different purposes: **understanding vs generation.**
- The real time Examples of generation of Text are Chat bots, Story generation and Code generation.

BERT Model:

- The word BERT abbreviated as “Bidirectional Encoder Representations from Transformers”.
- BERT is a powerful **Transformer based language model** developed by Google for understanding the context of text.
- BERT understands the meaning of a word using **both previous and next words**.
- For Example, let us take text “He went to the bank to withdraw money”, where BERT understands bank is financial institution.
- BERT(Bidirectional Encoder Representations from Transformers) is a machine learning model designed for natural language processing tasks, focusing on understanding the context of text.
- BERT is trained on large amounts of unlabeled text to learn contextual representations of words based on their surrounding context.
- BERT uses a transformer based encoder to process input text and generate contextualized representations for each token.

BERT Model:

- BERT uses a multilayer bidirectional Transformer encoder to understand text by capturing context from both directions.
- It Built on a stack of Transformer encoder layers using self-attention and feed forward networks.
- BERTBASE has 12 layers in the Encoder stack while BERTLARGE has 24 layers in the Encoder stack.



BERT Model:

- BERT takes a sentence as input for training.
- It randomly replaces **15% of the words** with the **[MASK]** token.
- The masked sentence is given to the **BERT model**.
- BERT reads the sentence from **both left and right directions**.
- It uses the surrounding words to understand the context.
- A **Softmax layer** predicts the original masked word.
- The prediction is compared with the correct word, and the model is updated.
- By repeating this process many times, **BERT learns the meaning and context of words**, making it useful for many NLP tasks.

Note:-BERT reads in **both directions at the same time** using its **self-attention mechanism**. Unlike traditional models that read text only from left-to-right or right-to-left, BERT allows every word to attend to every other word in the sentence during a single forward pass.

Example

Sentence:

The cat sat on the mat.

Suppose the word "sat" is masked:

The cat [MASK] on the mat.

To predict "sat", BERT looks at:

- **Left context:** "The cat"
- **Right context:** "on the mat"

It combines information from **both sides** to predict the missing word.

BERT Model:

- BERT applications:
- Sentiment Analysis: Evaluates customer reviews and social media to classify sentiment (positive/negative/neutral).
- Question Answering (QA) Systems: Identifies specific answers from a passage, commonly used in chat bots and virtual assistants.
- Search Enhancements: Improves search engine relevance by understanding user query context (used by Google).
- Named Entity Recognition (NER): Identifies entities like names, organizations, and locations in text.
- Text Classification: Organizes documents, labels topics, and filters spam.
- Summarization: Creates summaries by extracting key sentences or generating new, abstractive ones.
- Language Understanding in Legal/Finance : Analyzes contracts, extracts key clauses, and identifies risks.

GPT models:

- Generative Pre trained Transformer(GPT) is a language model that understands and generates human like text.
- They are designed to understand and generate human-like text based on input prompts.
- It learns patterns and relationships between words from large data and can perform multiple language tasks using a single model.
- GPT models are powerful tools in AI that enable natural language understanding and generation, widely used in modern applications.
- GPT(Generative Pre-trained Transformer) models are advanced language models developed by Open AI.
- They are designed to understand and generate human-like text using the Transformer architecture.
- GPT models are pre-trained on large text datasets and then fine-tuned for specific tasks.

GPT models:

- They use attention mechanisms to capture context and relationships between words in a sentence.
- The model predicts the next word in a sequence, enabling it to generate meaningful and coherent text.
- Key versions include GPT-3, GPT-3.5, and GPT-4, GPT-5 (latest) each improving in performance and accuracy.
- The word “GPT” includes Generative means Can generate text (answers, stories, code, etc.), pre-trained means Trained on large datasets before being used, Transformer means Uses the Transformer architecture for processing language.

GPT models:

- They use attention mechanisms to capture context and relationships between words in a sentence.
- The model predicts the next word in a sequence, enabling it to generate meaningful and coherent text.
- Key versions include GPT-3, GPT-3.5, and GPT-4, GPT-5 (latest) each improving in performance and accuracy.
- The word “GPT” includes Generative means Can generate text (answers, stories, code, etc.), pre-trained means Trained on large datasets before being used, Transformer means Uses the Transformer architecture for processing language.
- The GPT model working flow as given below step by step i.e.,
 - step1: Input text is broken in to tokens.
 - step2: Tokens are converted in to embeddings.
 - step3: The Transformer architecture processes the data using attention.
 - step4: The model predicts the next word repeatedly.
 - step5: Final output is generated as meaningful text

GPT models:

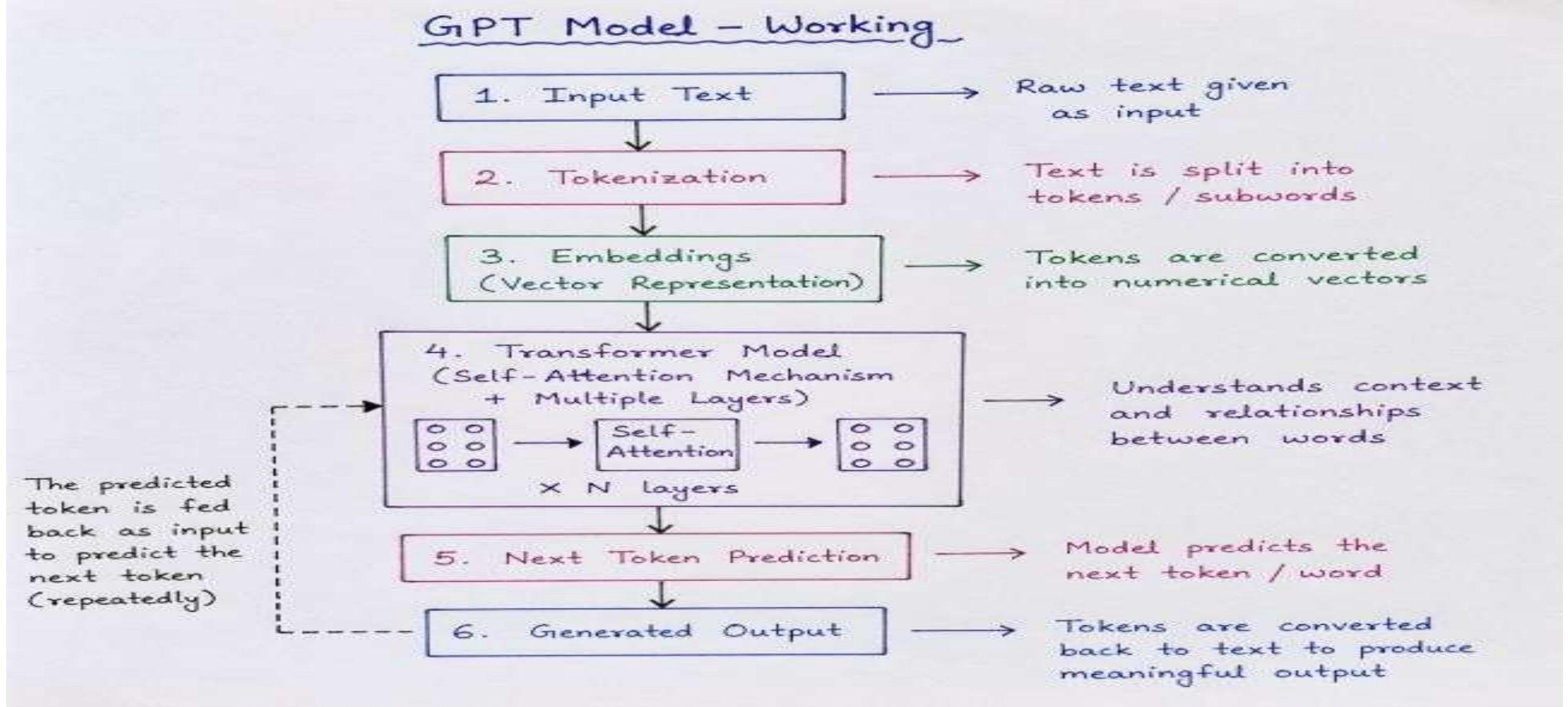


Fig.GPT Architecture working flow

GPT models:

- **Applications of GPT:**
 - ✓ **Content Generation & Marketing** like Creative Writing, Marketing Copy, Email Generation.
 - ✓ **Software Development & Coding** like Code Generation, Debugging & Documentation, Technical Support.
 - ✓ **Customer Service & Virtual Assistants** like Conversational Chat bots, Voice Assistants, Task Management.
 - ✓ **Data Analysis & Knowledge Management** like Summarization, Data Extraction & Structuring, Sentiment Analysis.
 - ✓ **Education & Learning** like Tutoring, Material Generation.
 - ✓ **Specialized Domains** like Healthcare, Finance, Legal.

Auto encoding

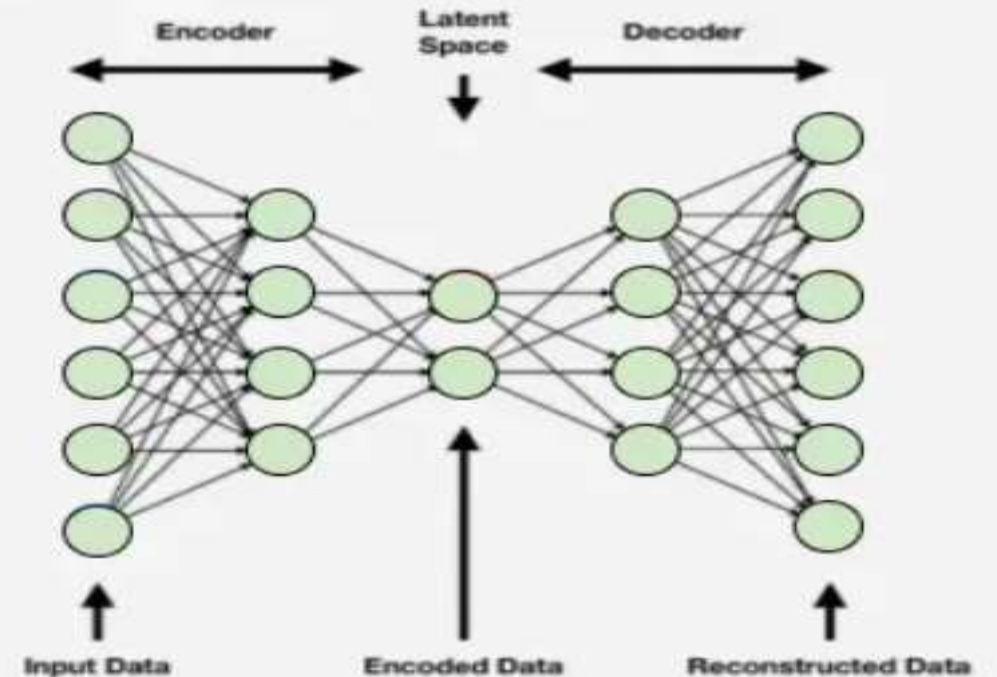
- Auto encoding models are powerful tools for learning compact and meaningful data representations, making them useful in many AI and machine learning applications.
- Auto encoders are a type of unsupervised, self-supervised neural network architecture designed to learn efficient representations of data by compressing (encoding) it and then reconstructing (decoding) it.
- They are primarily used for dimensionality reduction, feature extraction, data denoising, and anomaly
- Auto encoding models are a type of neural network used to learn efficient representations of data by compressing and reconstructing the input.
- They are widely used in deep learning for feature learning and data generation.
- Auto encoders are neural networks that compress input data into a smaller representation and then reconstruct it, helping the model learn important patterns efficiently.

AUTO ENCODER

- An auto encoder's architecture consists of three main components that work together to compress and then reconstruct data which are as follows:

What is an Autoencoder?

- Learns to compress and reconstruct data
- Two parts: Encoder (compress), Decoder (reconstruct)
- Used for noise removal, anomaly detection & feature extraction



AUTO ENCODER

1. Encoder

It compress the input data into a smaller, more manageable form by reducing its dimensionality while preserving important information. It has three layers which are:

Input Layer: Here the original data enters the network. It can be images, text features or any other structured data.

Hidden Layers: These layers perform a series of transformations on the input data. Each hidden layer applies weights and [activation functions](#) to capture important patterns, progressively reducing the data's size and complexity.

Output (Latent Space): The encoder outputs a compressed vector known as the latent representation or encoding. This vector captures the important features of the input data in a condensed form helps in filtering out noise and redundancies.

2. Bottleneck (Latent Space)

The bottleneck is the smallest layer in the network that holds a compressed representation of the input data. It forces the model to keep only the most important features, helping it learn key patterns and improve generalization.

AUTO ENCODER

3. Decoder

It is responsible for taking the compressed representation from the latent space and reconstructing it back into the original data form.

Hidden Layers: Progressively expand the latent vector back into a higher-dimensional space. Through successive transformations decoder attempts to restore the original data shape and details

Output Layer: Produces the reconstructed output which aims to closely resemble the original input. The quality of reconstruction depends on how well the encoder-decoder pair can minimize the difference between the input and output during training.

Applications

Data compression

Noise removal(image denoising)

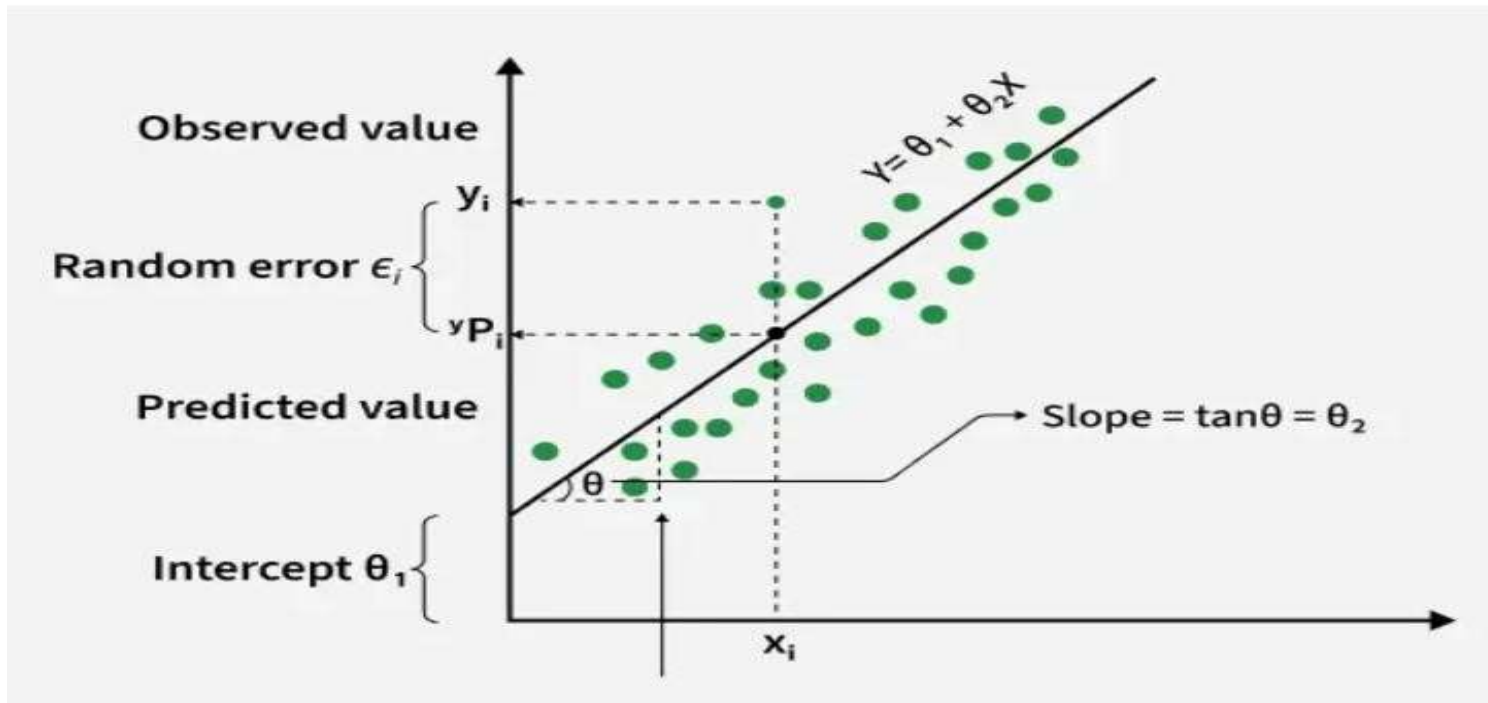
Feature extraction

Anomaly detection

Image generation

Regression Models:

- A Regression Model is a type of supervised learning algorithm used to predict continuous numerical values based on input data.
- It establishes a relationship between independent variables(inputs) and a dependent variable (output).
- The most common regression model is Linear Regression, which assumes a linear relationship between variables: $y = mx + c$



Where:

- ✓ y = predicted output
- ✓ x = input feature
- ✓ m = slope (relationship strength)
- ✓ c = intercept

Regression Models:

- Regression model architectures vary from simple linear equations to complex deep neural networks, with the common goal of predicting a continuous numerical target variable.
- These models establish relationships between independent variables (input features) and dependent variables(targets) to make predictions for forecasting, trend analysis, and numerical estimation.
- Regression models are essential for predicting continuous values and are widely used in real-world data analysis and machine learning tasks.
- Regression Model is trained by using labeled data and it Learns the relationship between input variables and output variables.
- It Minimizes the error between predicted and actual values and It Uses techniques like least squares method.
- The Applications of Regression models are House price prediction, Weather forecasting, Sales prediction and Stock market analysis.

Exploring Chat GPT:

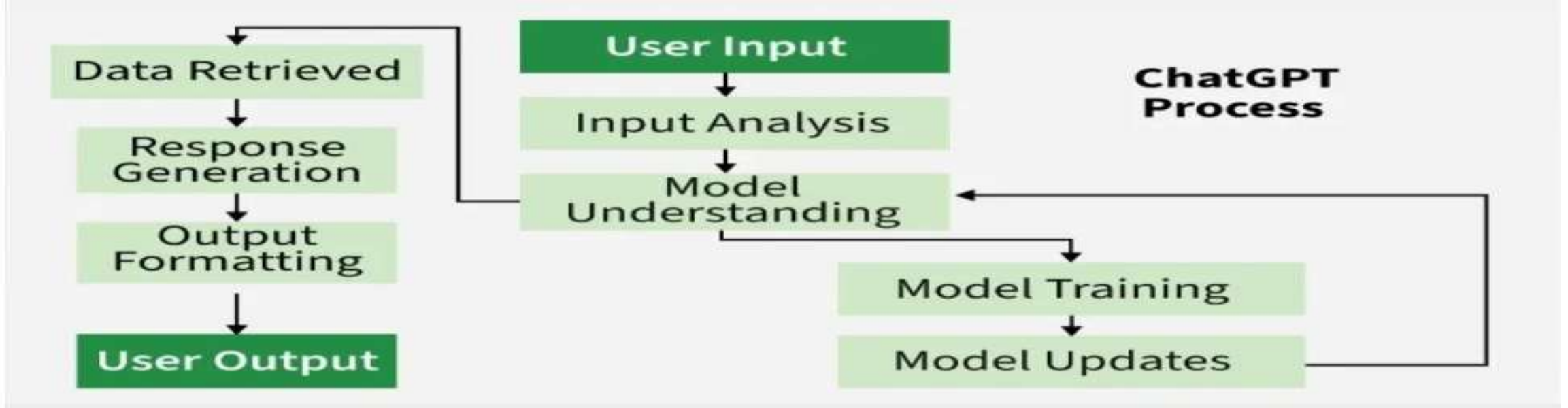
- Chat GPT is an AI chat bot developed by Open AI using the GPT model.
- GPT stands for **Generative Pre-trained Transformer**.
- Chat GPT uses advanced AI to understand and generate human-like text.
- It is built on the **Transformer** architecture.
- It understands questions and provides natural, meaningful responses.
- Chat GPT helps with learning, writing, coding, and problem-solving.
- It is widely used in education, business, research, and automation.
- Modern Chat GPT can also browse the web, analyze files, and generate images.

Exploring Chat GPT:

- **Natural Language Understanding (NLU):** Chat GPT understands and responds to human language naturally.
- **Versatility:** It can answer questions, write content, solve problems, and generate code.
- **Real-Time Responses:** It provides quick and instant responses to user queries.
- **Adaptability:** It improves its responses by learning from large amounts of training data and updates.

- **Description:-**

- By observing below diagram, we provide text-based input or a query to Input.
- It Processing and analyzes the text using its trained neural network to understand context and intent.
- Response Generation is Based on patterns learned from large datasets ; it generates a response that fits the context of the conversation.



- **Applications**
- Education and tutoring.
- Content writing.
- Programming assistance.
- Customer support.
- Research and summarization.

Prompt Engineering: Designing Prompts:

- **Prompt Engineering** is the process of creating effective prompts to get accurate and relevant responses from AI models.
- **A prompt** is the instruction or input given to an AI model to perform a specific task.
- **Prompt Engineering** helps improve the quality, accuracy, and usefulness of AI-generated outputs.
- **It involves designing**, testing, and refining prompts to achieve the desired results.
- **It provides clear context** and instructions, guiding AI to generate better responses.
- **Prompt Engineering** acts as a roadmap for communicating effectively with AI models.
- **Designing prompts** is a key part of Prompt Engineering for obtaining meaningful and reliable outputs.
- **Well-designed prompts** enable AI models like ChatGPT and Gemini to perform tasks more efficiently.
- **Effective Prompt Engineering** improves task performance by reducing ambiguity in instructions.
- **It is both an art and a science** of communicating with Large Language Models (LLMs).

DESIGNING PROMPTS

1. What is a Prompt?

A prompt is the input or instruction given to an AI model to perform a specific task and generate output.

Goal :
Get accurate,
relevant and
useful responses
from AI models.

3. Types of Prompts

Zero-shot Prompting

No examples are given.
(Simple instruction)

Few-shot Prompting

Includes few examples to guide the model.

Instruction-based Prompting

Direct instructions or commands are given.

Role-based Prompting

Assign a role to the model (e.g., teacher, expert, coder)

4. Example

Weak Prompt

Explain regression.

Well-Designed Prompt

Explain regression in 5 points with a formula and one real-world example.

6. Conclusion

Effective prompt design is key to getting high-quality, accurate and meaningful outputs from AI systems.

2. Principles of Designing Prompts

1. **Clarity and Precision**
– Use clear and specific language.
2. **Provide Context**
– Give necessary background information.
3. **Specify Output Format**
– Mention how the answer should be presented (e.g., points, table, diagram, etc.)
4. **Step-by-Step Instructions**
– Break complex tasks into smaller steps.
5. **Use Examples (Few-shot)**
– Provide sample inputs and outputs to guide the model.

5. Applications

- Content generation
- Chatbots
- Coding assistance
- Academic learning
- Data analysis



Tip :
Good prompts
lead to
better results!

Prompt Engineering: Designing Prompts:

- Techniques:

- Zero-shot Prompting : Asking for a result without examples.
- Few-shot Prompting : Providing a few examples(shots)to guide the model.
- Chain-of-Thought(CoT):Asking the model to explain its reasoning steps.
- Role Prompting: Assigning a person, e.g., "Act as a senior copy writer".

•

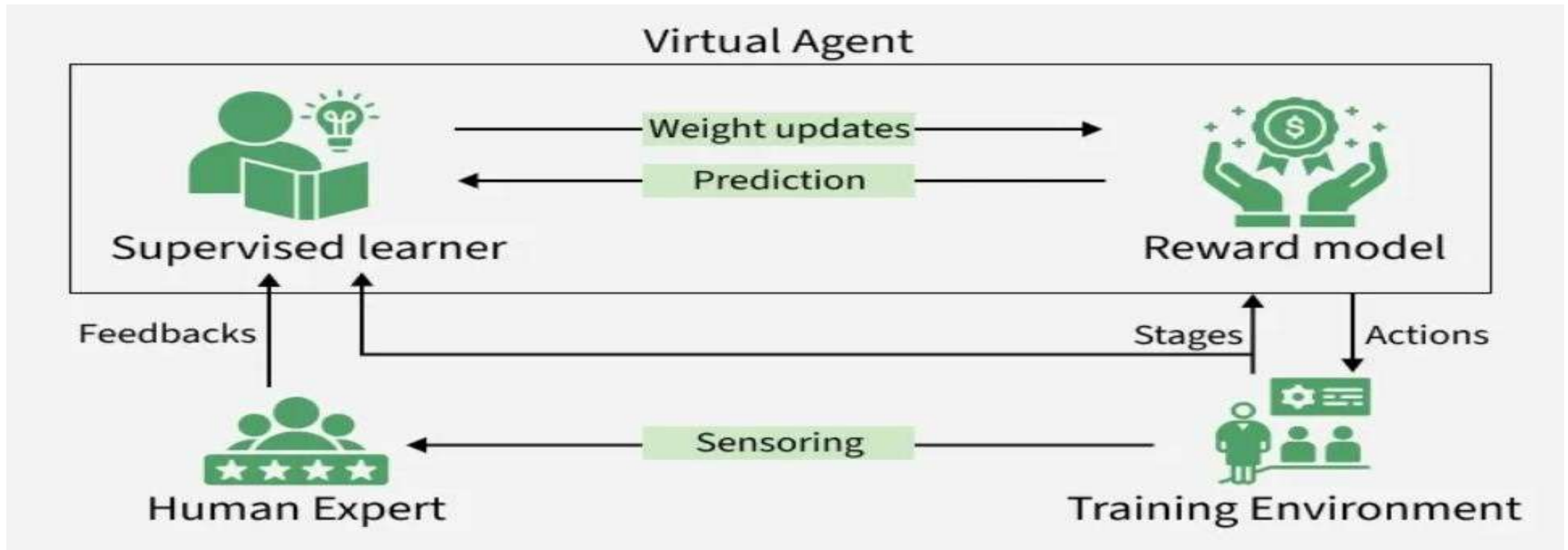
Applications

- Content generation
- Chat bots
- Coding help
- Academic learning
- Coding assistance
- Education

•

- **RLHF** is a technique that improves AI models using human feedback.
- It helps AI generate more accurate, safe, and useful responses.
- RLHF combines **Reinforcement Learning (RL)** with human evaluations.
- Human feedback is provided through ratings, rankings, or corrections.
- The AI learns from feedback to improve its future responses.
- RLHF aligns AI outputs with human preferences and expectations.
- It uses an iterative cycle of feedback, training, and refinement.
- RLHF optimizes Large Language Models (LLMs) for better performance.
- It reduces incorrect, biased, or unsafe AI-generated content.
- RLHF is widely used in Generative AI models such as **ChatGPT**.

The figure illustrates the **Reinforcement Learning from Human Feedback (RLHF)** training process, which is used to improve AI models such as ChatGPT. It shows how a **Virtual Agent**, **Human Expert**, **Reward Model**, and **Training Environment** work together to train the AI.



1. Virtual Agent (AI Model)

- The **Virtual Agent** is the AI model that learns to perform tasks.
- It generates **predictions or responses** based on the input it receives.
- During training, its **weights (parameters)** are continuously updated to improve its performance.

2. Supervised Learner

- Initially, the AI is trained using **supervised learning** with examples prepared by humans.
- It learns the correct way to answer questions or perform tasks.
- This provides the AI with a strong starting point before reinforcement learning begins.

3. Human Expert

- Human experts evaluate the AI's responses.
- They provide **feedback** by identifying which responses are accurate, helpful, or incorrect.
- This feedback is used to improve the AI's learning process.

4. Reward Model

- The **Reward Model** learns from the rankings and feedback given by human experts.
- It assigns a **reward score** to each AI response.
- Better responses receive higher rewards, while poor responses receive lower rewards.

5. Training Environment

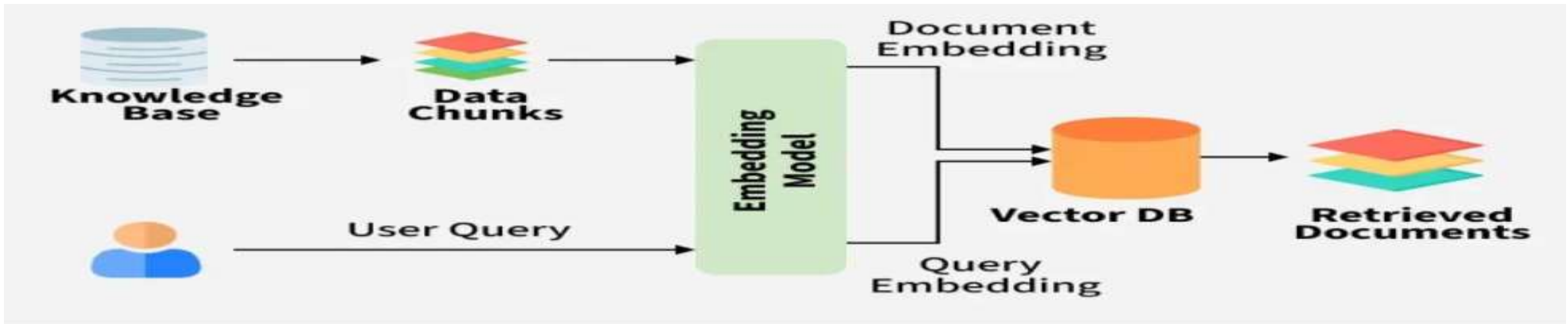
- The training environment presents different tasks or situations to the AI.
- The AI performs **actions** by generating responses.
- These actions are evaluated by the reward model.

6. Sensoring (Observation)

- The training environment provides observations or results back to the human experts.
- Human experts monitor the AI's behavior and continue giving feedback for improvement.

Retrieval Augmented Generation:

- Retrieval-Augmented Generation (RAG) is an AI technique that combines information retrieval and text generation.
- It first retrieves relevant information from external sources such as documents, databases, or the web.
- Then, it uses the retrieved information to generate an accurate and context-aware response.
- RAG helps AI models provide more accurate, relevant, and up-to-date answers. It is commonly used with large language models (LLMs) in Chat GPT.



Retrieval Augmented Generation:

- Working of RAG:
- The main components of RAG are:
- External Knowledge Source: Stores domain specific or general information like documents, APIs or databases.
- Text Chunking and Preprocessing: Breaks large text in to smaller, manageable chunks and cleans it for consistency.
- Embedding Mode I: Converts text into numerical vectors that capture semantic meaning.
- Vector Database : Stores embeddings and enables similarity search for fast information retrieval.
- Query Encoder : Transforms the user's query into a vector for comparison with stored embeddings.
- Retriever : Finds and returns the most relevant chunks from the database based on query similarity.
- Prompt Augmentation Layer: Combines retrieved chunks with the user's query to provide context to the LLM.

Retrieval Augmented Generation:

- LLM(Generator): Generates a grounded response using both the query and retrieved knowledge.
- Updater(Optional): Regularly refreshes and re-embeds data to keep the knowledge base up to date.
- The system first searches external sources for relevant information based on the user's query instead of relying only on existing training data.
- RAG is nothing but Retrieval of information + Generation of response.
- The below diagram shows how Retrieval-Augmented Generation(RAG) enhances AI responses by combining retrieval and generation.



Retrieval Augmented Generation:

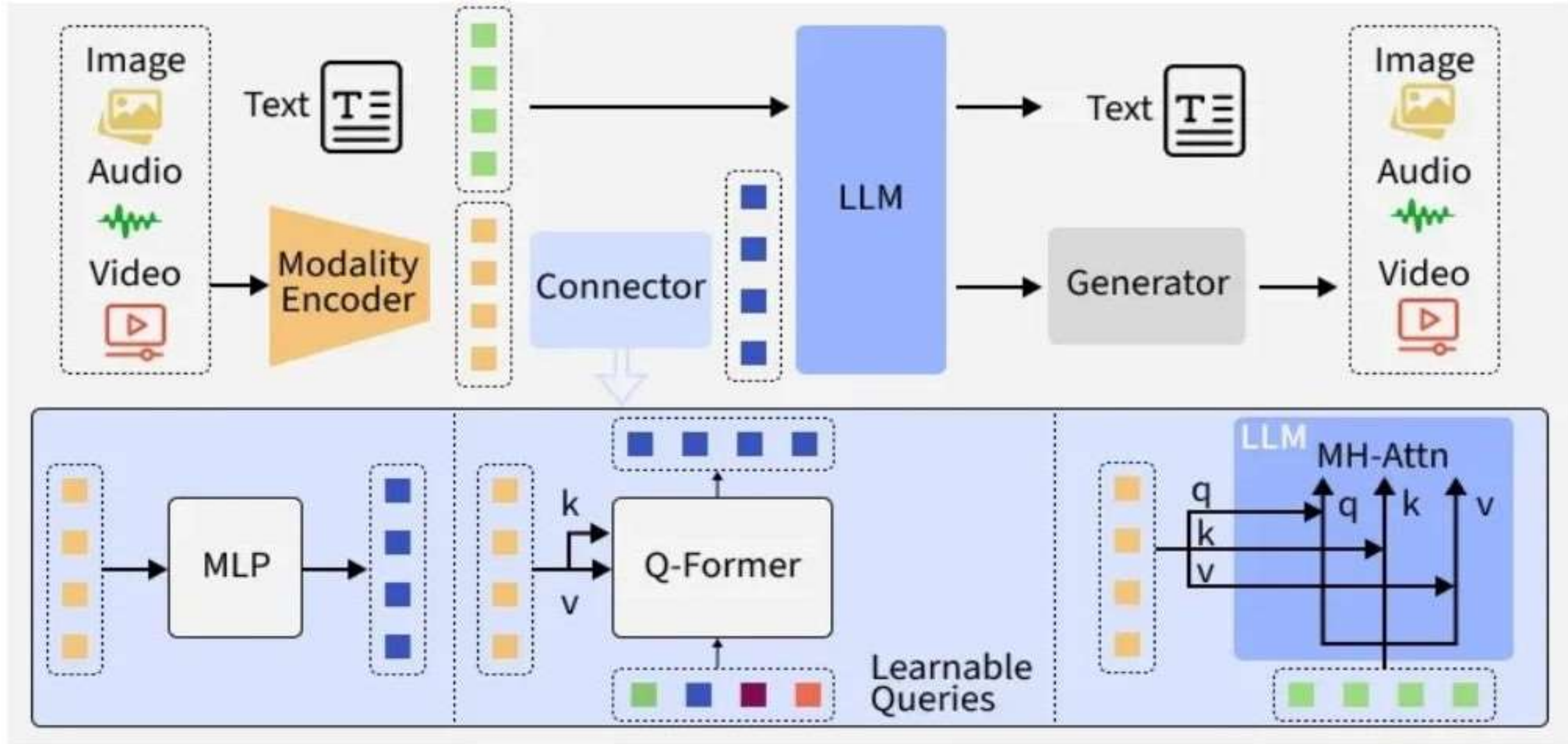
Step-by-Step Flow of RAG Architecture

- User → Prompt-The user gives a query(input prompt).
- Prompt → Data Retrieval-The system searches relevant information from large data sources (databases, documents, web).
- Data Retrieval Process –Search → Finds relevant data
- Retrieve → Extracts useful information
- Generator(LLM)-The retrieved data is passed to the generator(like Chat GPT)
- The model uses this context to generate an answer.
- Response-Final output is accurate and which is context-based.
- **Applications**
 - Question answering systems
 - Chat bots
 - Search engines
 - Document-based AI assistants

Multimodal LLM:

- A Multimodal Large Language Model (MLLM) is an advanced AI model that can understand and process text, images, audio, and video together.
- Multimodal Large Language Models (MLLMs) are AI systems designed to process, interpret, and generate information from multiple data types instead of only text.
- MLLMs extend traditional language models by integrating different types of input, making them more powerful and versatile.
- They combine information from text, images, audio, and video to better understand context and provide more accurate responses.
- MLLMs process multiple data formats simultaneously to improve understanding and generate meaningful outputs.
- For example, an MLLM can read a text description, analyze a related image, and produce a response using both inputs.
- Popular examples of MLLMs include GPT-4, which is used in applications such as Chat GPT.

Multimodal LLM:



Architecture of a Multimodal Large Language Model (MLLM)

- The model accepts different types of inputs such as **text, images, audio, and video**.
- A **Modality Encoder** converts images, audio, and video into numerical feature representations (embeddings).
- Text input is directly converted into text embeddings for processing.
- A **Connector** aligns the encoded multimodal features with the language model's input format.
- The **LLM (Large Language Model)** combines text and multimodal features to understand the overall context.
- The LLM processes the combined information and generates meaningful responses.
- The output can be **text** or other modalities such as **images, audio, or video** using a generator module.

Overall Working Flow

Step 1: Receive input in the form of text, image, audio, or video.

Step 2: Encode non-text data using a Modality Encoder.

Step 3: Convert encoded features into LLM-compatible embeddings through a Connector.

Step 4: Feed both text and multimodal embeddings into the LLM.

Step 5: The LLM understands the combined information and generates a response.

Step 6: A Generator converts the response into text, image, audio, or video, depending on the task.

Types of Connectors :

Types of Connectors Used in MLLMs

MLP (Multi-Layer Perceptron): Maps multimodal features into the embedding space of the LLM using a simple neural network.

Q-Former (Query Transformer): Uses learnable query tokens to extract the most relevant information from multimodal features before sending them to the LLM.

Cross-Attention Connector: Enables the LLM to focus on important multimodal features by using attention mechanisms (Query, Key, and Value).

Key Components:



1. Data integration



2. Feature Extraction

Key Components



3. Cross-Model
Attention



4. Joint Representation

• Key Components:

Components of Multimodal Large Language Models

- **Data Integration:** Combines data from multiple sources into a single unified representation.
- **Feature Extraction:** Extracts important features from each input modality for effective processing.
- **Joint Representation:** Creates a unified representation of multimodal data to improve understanding and decision-making.
- **Cross-Modal Attention:** Focuses on the most relevant information across different modalities to generate accurate and context-aware responses.

• Applications

- Image captioning
- Visual question answering
- Speech recognition
- Video analysis
- Medical diagnosis
- Health care
- Education
- Customer Service

Issues of LLM like hallucination:

- **Ambiguous Prompts:** Unclear or confusing questions can cause the model to generate incorrect answers.
- **Knowledge Gaps:** The model may give wrong information when it lacks knowledge about a topic.
- **False Information:** The model can produce incorrect or fabricated facts that appear to be true.
- **Loss of Trust:** Frequent incorrect answers reduce users' confidence in the AI system.
- **Misleading Decisions:** Incorrect outputs can lead to poor decisions in areas like healthcare, education, and finance.
- **Lack of Explainability:** The model often cannot explain why it generated a particular answer.
- **Inconsistent Responses:** The same question may receive different answers at different times.
- **Bias Amplification:** The model may repeat or strengthen biases present in its training data.
- **Training Data Errors:** Mistakes or missing information in the training data can cause hallucinations.
- **Overconfidence:** The model may present false information with high confidence, making it difficult to identify errors.

Thank You